PROJECT SYNOPSIS

“AGILEGRAPH”

Graph-Learned Crypto-Agility Risk Scoring for Post-Quantum Migration

Students: 1. Spoorthi — NNM23CB058 | 2. Sindhoora R — NNM23CB057 | 3. Nivedita V Shetty — NNM23CB033 | 4. Sujan S — NNM23CB062

Guide Name: Ms Madhushree

Department: Department of Computer Science (Cyber Security)

Institution: NMAM Institute of Technology (NMAMIT), Nitte (Deemed to be University), Karnataka

Academic Year: 2023 – 27 (Final Year, B.Tech)

1. Introduction

The security of most digital systems in use today, ranging from online banking and government communications to the Internet itself, rests on public-key cryptosystems such as RSA and ECC. Quantum computers large and robust enough to break these cryptosystems with Shor’s algorithm have not yet been built, but the threat is more than a possibility: adversaries can already capture encrypted data and decrypt it once sufficiently powerful quantum computers become available, an approach known as “harvest-now, decrypt-later.” To counter this threat, NIST has published its first finalized post-quantum cryptography standards: ML-KEM (FIPS 203), ML-DSA (FIPS 204), and SLH-DSA (FIPS 205).

However, having quantum-resistant algorithms available does not make migration straightforward. Recent studies show that few organizations are able to carry out a crypto-agility assessment or maintain an accurate cryptographic inventory, both of which are prerequisites for a proper migration [1], [2]. Several migration strategies and roadmaps have been proposed at the enterprise and industry level [3]–[5]; however, turning a roadmap into a per-asset migration plan still requires an organization to first understand how cryptography is actually used within its software. AgileGraph fills the gap between discovery and prioritization by combining automated cryptographic discovery with graph modeling to rank cryptographic risks.

2. Problem Statement

While NIST has standardized post-quantum cryptographic algorithms, organizations still need practical processes and tools to discover their existing cryptographic footprint and translate it into asset-level migration priorities. The first step is identifying every instance of quantum-vulnerable cryptography in an organization's software. This is difficult because the cryptographic footprint is spread across legacy code, libraries, certificates, and endpoint devices, and organizations often have only a limited view of its extent. Identification is still largely manual, relying on analysis of code, certificates, and network traffic.

Even when cryptography is identified, existing tools report it as a flat list. They can point out that "a certificate has expired," but not that "updating this component will protect our most sensitive data with the least migration effort." There is a need for an auditable, explainable method that ranks cryptographic assets by taking into account the relationships between them.

3. Objectives

- Develop an automated cryptographic discovery pipeline for Python, Java, and Go source code and dependency manifests, and represent the discovered assets in a typed heterogeneous risk graph with file-and-line provenance, evaluated for per-language precision and recall on a seeded ground-truth benchmark and checked for provenance integrity on open-source projects.
- Define an auditable, factor-decomposed heuristic risk score with explicit missing-data handling, and verify its robustness through weight-perturbation rank-stability analysis (with bootstrap confidence intervals) and standards-anchored consistency and metamorphic tests.
- Implement a GATv2-based risk-propagation model over a graph-aware target using leave-one-project-out evaluation, and quantify, through a leakage ablation with feature-only, structure-only, and edge-shuffle controls, how much of the target is recoverable from graph structure.
- Benchmark flat, rule-based, heuristic, and GNN-based rankings using rank-agreement metrics with bootstrap confidence intervals, and a predefined structural scenario suite that measures adherence to stated dependency-aware prioritization rules.
- Deliver a command-line interface and web dashboard presenting a ranked migration list, factor-level explanations, a risk-reduction estimate, and a configurable Mosca Readiness Index, verified through automated API, interface, and deterministic-

reproducibility tests.

4. Literature Survey

4.1 Existing System

Modern cryptographic discovery is typically done through manual audit or with commercial discovery and inventory tools such as Keyfactor and AppViewX, which record certificates and keys but do not model how a weakness in one element affects the others. Studies of crypto-agility maturity indicate that most organizations are still at an early stage of replacing cryptographic primitives without extensive code changes [1], and enterprise- and sector-level roadmaps have been proposed to guide migration, for example for the financial industry [3], [5]. The Cryptography/Software Bill of Materials (CBOM/SBOM) concept standardizes the machine-readable recording of such inventories, although design and adoption issues remain under active study [12], [13].

4.2 Limitations of Existing System

- Discovered artifacts are recorded as cryptographic assets, but the output is flat and does not prioritize them or capture their downstream impact.
- The risk-score formulas used in commercial products are closed and vendor-dependent, which makes them impossible to audit.
- Existing solutions generally do not model dependencies between components, such as the fact that migrating one library or key may affect downstream systems.
- Few solutions explain why a particular artifact is dangerous, leaving the process a black box for the security team.
- Existing approaches provide limited support for migration ordering and for comparing alternative rankings.
- Where a risk formula exists, it is treated as ground truth, leaving no room for a learning model to build on it.
- There is limited published work that directly compares a heuristic risk formula with graph-based propagation.

4.3 Related Research Papers

The first group of papers describes the difficulty of cryptographic change and migration planning. Marchesi, Marchesi, and Tonelli survey crypto-agility and agile practices in light of quantum resistance and show that most organizations lack the structure needed for rapid cryptographic change [1]. Näther et al. systematically review the literature on software migration toward post-quantum cryptography and describe existing approaches together with gaps in migration tooling and automation [2]. Campbell reviews enterprise migration timelines and proposes models for sequencing PQC adoption [3]. Cherkaoui Dekkaki, Tasic, and Cano review the PQC transition process and identify areas that need dedicated tooling [4]. Kuka et al. describe global roadmaps toward the post-quantum era in finance and present a migration playbook for that sector [5].

The second group covers graph-based approaches to security risk evaluation and motivates graph-based learning in AgileGraph. Yin et al. propose a compact vulnerability knowledge graph for risk assessment, showing the advantages of a graph over a flat list of vulnerabilities [6], and investigate co-exploitation behavior of software vulnerabilities through knowledge-driven cybersecurity intelligence [7]. Automatic detection of cryptographic API misuse is addressed for Java by Zhang et al. [8] and for Python by Frantz et al. [9]; each addresses detection within a single programming language. Yuan et al. [10] and Xuan, Luong, and Thanh [11] apply graph neural networks to vulnerability detection in Go and in source code generally. Beyond source code, Kosasih et al. apply graph neural networks to supply-chain risk reasoning [14], Zhang et al. use graph neural networks with attention mechanisms for anomaly-node detection in network security [15], and Ahanger et al. apply graph attention networks to intrusion detection in IoT settings [16]. Together these works support the choice of a graph-attention architecture (GATv2) for propagating risk across the cryptographic asset graph in this project.

The third group concerns inventory standards. Bi et al. analyze design issues and solutions in the practical use of SBOMs [12], while McGraw, Migues, and Wheeler argue that SBOMs are both necessary and still immature [13]. This motivates AgileGraph's graph-based inventory, which extends the plain bill-of-materials concept with relationships between assets and with risk scoring. Because the weights of any risk formula are a design choice rather than a measured quantity, AgileGraph treats them as an initial, configurable estimate and examines how sensitive the resulting ranking is to them (Section 5.4).

5. Proposed System

AgileGraph consists of four interacting layers. The **scanning and data collection layer** analyzes source code through static analysis, using abstract-syntax-tree analysis for Python and pattern-based analysis for Java and Go, to detect cryptographic primitives such as RSA, ECDSA, and AES. It also parses dependency manifests to identify the cryptographic libraries in use, including deprecated or unsupported ones. Every finding carries file-and-line provenance, and the results are stored in a graph (NetworkX) for querying and reasoning about relationships between assets.

The **risk-graph construction layer** builds a typed, directed heterogeneous graph. The schema defines six node types: files, code locations where cryptography is used, certificates, network endpoints, libraries, and sensitive data. Typed relationships connect them (for example, a file contains a cryptographic usage, and a file depends on a library). Source scanning populates the file, cryptographic-usage, and library nodes; the remaining node types are defined in the schema for context-supplied data and future scanners.

The **scoring and learning layer** assigns every file-level asset an initial heuristic risk value from an auditable formula that combines seven measurable factors: data sensitivity, asset criticality, Internet exposure, strength of the cryptographic algorithm, CVE risk, library centrality, and migration difficulty. Factors that cannot be derived from source code alone (sensitivity, criticality, exposure, and CVE information) are accepted as declared context; when a factor is unavailable it is recorded as missing and the score is renormalized over the available factors, so missing information is never interpreted as zero risk. The formula makes no assumption about the arrival date of large-scale quantum computers. A GATv2 model then studies how risk propagates across the graph (Section 5.1). The **presentation layer** is described in Section 5.7.

5.1 Weak Supervision, the Role of the GNN, and the Leakage Ablation

An important design choice in AgileGraph is that the heuristic risk score is treated as an initial estimate, not as the true answer. Because the local heuristic is a weighted sum of node-level factors, a model that receives those factors as input can reproduce it exactly without using any graph structure, so removing only the final score from the inputs would not show that graph propagation contributes anything. The study therefore defines the learning target as a graph-aware score: the local heuristic score plus a transparent propagation term through which a file inherits risk from the libraries it depends on, weighted by their centrality. The propagation term and its components are excluded from the model inputs, so the only route by which the model can learn it is message passing over the graph.

The model is trained and tested with leave-one-project-out evaluation, so that it is always evaluated on projects it has not seen, in the following configurations:

- **(a)** GATv2 using local node features and graph edges;
- **(b)** a multilayer perceptron using the same local features without edges;
- **(c)** GATv2 with the edges randomly shuffled;
- **(d)** a structure-only GATv2 that sees only node types and topology;
- **(e)** a deliberate-leakage reference in which the final heuristic score is supplied as an input, included to show why it is excluded from the main configuration.

The contrasts (a)–(b) and (a)–(c) indicate how much of the target is recoverable through graph edges rather than through node features alone. Results are reported as Spearman rank correlation, top-k overlap, and macro-F1 over High/Medium/Low tiers derived from score thresholds, with cluster-bootstrap confidence intervals over projects, in a separate ablation table. These measures describe how well the structure-aware score is recovered; they are not a measure of accuracy against an independent ground truth.

The project then compares four rankings: a flat severity-only baseline (ranking by algorithm weakness alone, representing inventory-style tools), a rule-based baseline, the heuristic ranking, and the GNN-refined ranking. Differences are quantified with Kendall's τ, rank displacement, top-k overlap, and tier confusion matrices, with bootstrap confidence intervals. Specific case studies show where graph-aware rankings reorder assets relative to the flat baseline because of relational structure that an isolated score cannot see.

5.2 Robustness and Verification

No independent asset-level ground truth for real-world cryptographic migration risk exists. The evaluation is therefore designed to measure what can be measured rigorously: correctness of the implementation, stability of the ranking, and the effect of graph structure. It comprises five automated checks.

- **Scanner benchmark.** Seeded fixtures with known, planted cryptographic usages in Python, Java, and Go are scanned, and precision and recall are reported per language and per API pattern. Patterns that the scanners do not recognize are documented explicitly.
- **Provenance integrity.** An automated script verifies that the evidence snippet of every finding, across the open-source corpus, resolves at the recorded file and line.

- **Consistency and metamorphic tests.** Table-driven tests anchored to published NIST post-quantum transition guidance check the algorithm-strength ordering; further tests check monotonicity (increasing a factor never decreases the score), equivalence of the same cryptographic usage across languages, invariance to file renaming, correct behavior under missing data, and identical output on repeated scans.
- **Structural scenario suite.** Predefined scenarios with expected orderings are run against every ranking method, for example an identical weak algorithm in a widely depended-upon file versus an isolated file, a library with many dependents versus one with few, and cases with missing factors. Because the expected orderings are stated design rules of the method, the suite verifies that the system implements its stated prioritization semantics; it does not prove that those semantics are correct.
- **Weight sensitivity analysis.** Described in Section 5.4.

The open-source corpus consists of at least eight publicly available projects, with at least two per language and spanning two size tiers. Project commit identifiers are recorded for reproducibility.

5.3 Mosca Readiness Index

Since the timeframe for a cryptographically relevant quantum computer cannot be scientifically quantified, it is excluded from the risk score entirely. AgileGraph instead provides a separate Mosca Readiness Index, derived from Mosca's inequality ($x + y > z$), where x is the required data confidentiality period, y is the estimated migration time, and z is the assumed quantum capability horizon. The index is presented as a configurable strategic planning metric on the dashboard, with z supplied as an adjustable parameter rather than a fixed constant, so that no unscientific assumption is built into the system. The index is kept outside the training of the model and is verified with boundary-value and monotonicity tests.

5.4 Weight Justification

The weights of the heuristic formula are an initial, documented, and configurable starting point; they are not claimed to be optimal. To show how much the conclusions depend on them, a sensitivity analysis perturbs the weights by approximately ±10% and ±20% over a large number of random draws, renormalizing after each draw. The resulting rankings are compared with the baseline ranking using Kendall's τ, Spearman correlation, and top-k overlap, with bootstrap confidence intervals. A drop-one-factor analysis additionally identifies which factors dominate the ranking. The factors whose perturbation reorders

the highest-priority assets are reported explicitly. This is a scoped and defensible justification appropriate to a B.Tech project timeline, and it is named as a limitation in the final writeup.

5.5 Explainability Mechanism

Plain-language explanations are generated from two combined sources: (1) a decomposition of the heuristic factors (data sensitivity, exposure, CVE risk, algorithm strength, centrality, effort) that shows which measurable inputs dominate the base score, and (2) a breakdown of the propagation term showing which dependent libraries and relationships raised an asset's priority. For the trained model, GATv2 attention coefficients are used as supplementary evidence of which neighboring nodes influenced a prediction. The dashboard presents these together, so that a security team is not given a single opaque number without a stated mechanism.

5.6 Scanning Scope and Authorization

AgileGraph analyzes publicly available open-source code and code owned by the team, and does not probe third-party infrastructure. Any network-facing analysis is limited to infrastructure controlled by the team and for which written permission has been obtained. Scanning third-party infrastructure without permission raises legal and ethical issues, and the methodology therefore states this authorization boundary explicitly.

5.7 Presentation Layer

The presentation layer consists of a command-line interface that produces deterministic scan reports and a FastAPI backend with a React-based web dashboard. A user scans a project and receives a ranked migration to-do list, each item annotated with a plain-English explanation and an estimate of the overall risk reduced by addressing it, computed by recomputing the aggregate score with the asset replaced by a post-quantum-safe alternative. The separate Mosca Readiness Index view is provided alongside. Because this estimate is calculated within AgileGraph's own risk model, it supports prioritization rather than serving as an independent measure of real-world risk.

5.8 Scope and Limitations

- Cryptographic discovery is pattern-based: recall is bounded by the recognized API patterns, and results are reported only within them. Certificate, endpoint, and sensitive-data discovery are outside the current scanners.

- Risk scores are not validated against independent ground truth, such as real migration outcomes or incident data. Evaluation therefore concerns implementation correctness, ranking stability, and the contribution of graph structure.
- The GNN is a delimited research extension. Its target is a model-defined graph-aware score, so its results show how much of that score is recoverable from structure, not that it is more correct than the heuristic.
- With a corpus of open-source projects, statistical results are exploratory and are reported as effect sizes with confidence intervals.
- Heuristic weights are an initial, configurable estimate whose influence is examined through sensitivity analysis.

6. System Requirements

6.1 Hardware Requirements

- Development workstation or laptop with a minimum of 16 GB RAM and a multi-core processor.
- GPU (NVIDIA, CUDA-compatible) recommended for accelerated GNN training, though not strictly required for the target dataset scale.
- Internet connectivity for retrieving open-source projects and installing dependencies.

6.2 Software Requirements

- Operating System: Linux / Windows with WSL2 / macOS
- Programming Languages: Python 3.10+, JavaScript/TypeScript
- AI/ML Libraries: PyTorch, PyTorch Geometric (GATv2Conv)
- Code Analysis: Python `ast` module and custom pattern-based analyzers for Java and Go; dependency-manifest parsers
- Graph Storage: NetworkX
- Backend Framework: FastAPI
- Frontend Framework: React with Tailwind CSS
- Evaluation Tools: scikit-learn (macro-F1), SciPy and NumPy (Kendall's τ, Spearman correlation, bootstrap confidence intervals), Matplotlib
- Testing: pytest and a frontend test suite, with GitHub Actions for continuous integration
- Version Control: Git / GitHub

7. Expected Outcomes

- A prototype system that scans a codebase and produces a ranked, explainable list of PQC migration candidates.
- A publicly available, well-documented risk-scoring heuristic justified by sensitivity analysis, and a heterogeneous risk graph populated from at least eight open-source projects across three languages and two size tiers.
- A scanner accuracy report based on a seeded ground-truth benchmark, together with an automated provenance-integrity check.
- A GATv2-based risk-propagation study with a leakage ablation (feature-only, structure-only, and edge-shuffle controls) under leave-one-project-out evaluation.
- A quantitative comparison of four ranking methods (flat, rule-based, heuristic, and GNN-refined) using rank-agreement metrics with bootstrap confidence intervals and a structural scenario suite.
- A command-line interface and web dashboard covering the pipeline from scanning to the ranked migration list, with factor-level explanations, a risk-reduction estimate, and a Mosca Readiness Index view.
- A paper draft describing the graph model, the risk-scoring heuristic, and the evaluation results.

8. Applications

- Post-quantum cryptography migration strategy development for industries such as finance, healthcare, telecommunications, and government.
- Tools for Security Operations Center (SOC) and Governance-Risk-Compliance (GRC) teams supporting ongoing cryptography risk assessment.
- Cybersecurity consulting services where a defensible, justifiable migration plan is required.
- Use in academia and research as a transparent, verifiable counterpart to proprietary vendor risk-scoring algorithms.
- Supply-chain security, extending current Software Bill of Materials (SBOM) approaches with cryptography-specific analysis [12], [13].

9. Future Scope

- Extension of scanner support to new languages (C/C++, Rust, JavaScript), cloud-native and infrastructure-as-code ecosystems, and certificate, TLS, and Certificate Transparency analysis.
- Extraction of endpoint and sensitive-data nodes, and export of results as CBOM documents for interoperability with existing inventory tooling.
- Transition from one-off snapshot scans to continuous monitoring through SIEM/SOAR integration.
- Incorporation of new PQC schemes (HQC, for example) and future editions of NIST guidelines.
- Modeling of several migration strategies at the enterprise level, taking budget and personnel limitations into account.
- Co-development of the tool with an industrial partner to test the risk-scoring approach against real-world incident and migration-outcome data.

10. Project Timeline

Phase	Activity	Deliverable
Phase 1	Planning + Design	Graph schema finalized; heuristic risk-scoring formula and target definition agreed with the guide; evaluation plan and scenario suite defined.
Phase 2	Building the Scanners	Code and dependency scanners for Python, Java, and Go; seeded benchmark and provenance check; risk graph populated from at least eight open-source projects across three languages and two size tiers.
Phase 3	Heuristic Scoring + Robustness	Heuristic scores computed; consistency and metamorphic tests; weight-perturbation and drop-one-factor sensitivity analysis.

Phase	Activity	Deliverable
Phase 4	Training the AI Model	GATv2 trained on the graph-aware target under leave-one-project-out evaluation; flat and rule-based baselines built; leakage ablation with feature-only, structure-only, and edge-shuffle controls.
Phase 5	Benchmarking + Dashboard	Four-method ranking comparison and scenario-suite results; web dashboard and command-line interface with explanations, risk-reduction estimate, and Mosca Readiness Index view.
Phase 6	Writeup	Final results, ablation tables, case studies of graph-aware reorderings, limitations section, and paper draft completed.

11. References

[1] L. Marchesi, M. Marchesi, and R. Tonelli, “A Survey on Cryptoagility and Agile Practices in the Light of Quantum Resistance,” Information and Software Technology, vol. 175, 2024.

[2] C. Näther, D. Herzinger, S.-L. Gazdag, J.-P. Steghöfer, S. Daum, and D. Loebenberger, “Migrating Software Systems Toward Post-Quantum Cryptography—A Systematic Literature Review,” IEEE Access, vol. 12, pp. 129064–129089, 2024.

[3] R. Campbell, “Enterprise Migration to Post-Quantum Cryptography: Timeline Analysis and Strategic Frameworks,” Computers, vol. 14, no. 1, Art. no. 9, 2025.

[4] K. Cherkaoui Dekkaki, I. Tasic, and M.-D. Cano, “Exploring Post-Quantum Cryptography: Review and Directions for the Transition Process,” Technologies, vol. 12, no. 12, Art. no. 241, 2024.

[5] C. Kuka, S. Muhyaddin, P. L. Teh, and L. Davies, “Global Roadmaps for Post-Quantum Era in Finance: Policies, Timelines, and a Pragmatic Playbook for Migration,” Risks, vol. 14, no. 2, 2026.

[6] J. Yin, W. Hong, H. Wang, J. Cao, Y. Miao, and Y. Zhang, “A Compact Vulnerability Knowledge Graph for Risk Assessment,” ACM Transactions on Knowledge Discovery from Data, vol. 17, no. 4, 2023.

[7] J. Yin, M. Tang, J. Cao, M. You, H. Wang, and M. Alazab, “Knowledge-Driven Cybersecurity Intelligence: Software Vulnerability Coexploitation Behavior Discovery,” IEEE Transactions on Industrial Informatics, vol. 19, no. 4, pp. 5593–5604, 2023.

[8] Y. Zhang, M. Kabir, Y. Xiao, D. Yao, and N. Meng, “Automatic Detection of Java Cryptographic API Misuses: Are We There Yet?,” IEEE Transactions on Software Engineering, vol. 49, no. 11, 2023.

[9] M. Frantz, Y. Xiao, T. S. Pias, N. Meng, and D. Yao, “Methods and Benchmark for Detecting Cryptographic API Misuses in Python,” IEEE Transactions on Software Engineering, vol. 50, no. 5, pp. 1118–1142, 2024.

[10] L. Yuan, Y. Fang, Q. Zhang, Z. Liu, and Y. Xu, “Go Source Code Vulnerability Detection Method Based on Graph Neural Network,” Applied Sciences, vol. 15, no. 12, Art. no. 6524, 2025.

[11] C. D. Xuan, T. T. Luong, and M. C. Thanh, “Optimising Source Code Vulnerability Detection Using Deep Learning and Deep Graph Network,” Connection Science, vol. 37, no. 1, 2025.

[12] T. Bi, B. Xia, Z. Xing, Q. Lu, and L. Zhu, “On the Way to SBOMs: Investigating Design Issues and Solutions in Practice,” ACM Transactions on Software Engineering and Methodology, vol. 33, no. 1, 2023.

[13] G. McGraw, S. Migues, and D. A. Wheeler, “Software Bills of Materials Are Required. Are We There Yet?,” IEEE Security & Privacy, vol. 21, no. 2, pp. 82–87, 2023.

[14] E. E. Kosasih, F. Margaroli, S. Gelli, A. Aziz, N. Wildgoose, and A. Brintrup, “Towards Knowledge Graph Reasoning for Supply Chain Risk Management Using Graph Neural Networks,” International Journal of Production Research, vol. 62, no. 15, pp. 5596–5612, 2024.

[15] D. Zhang, J. Wang, H. Gao, Z. W. Ni, and H. Zhang, “Network Security Anomaly Node Detection Based on Graph Neural Network and Attention Mechanism,” Journal of Network and Systems Management, vol. 33, Art. no. 87, 2025.

[16] A. S. Ahanger, S. M. Khan, F. Masoodi, and A. O. Salau, “Advanced Intrusion Detection in Internet of Things Using Graph Attention Networks,” Scientific Reports, vol. 15, 2025.